DEEP AGENTS FROM ZERO

शून्यबाट **Deep Agents** सिक्ने व्यावहारिक पुस्तक

Beginner-friendly English + Nepali • Python • LangChain • LangGraph • Deep Agents

From “What is an LLM?” to tools, memory, skills, subagents, context engineering, MCP, human approval, debugging, and production architecture.

Documentation baseline: LangChain / LangGraph / Deep Agents docs verified 3 October 2026

Edition 1.0

Preface | यो पुस्तक कसका लागि हो?

This book assumes you are starting at level 0. You do not need to understand agents, LangChain, LangGraph, vector databases, middleware, or multi-agent systems before reading it. Basic Python helps, but important Python ideas are explained when they first appear.

> नेपालीमा: यो पुस्तक Deep Agent बारे केही पनि नजानेको व्यक्तिले पनि पढ्न मिल्ने गरी बनाइएको हो। Python को सामान्य ज्ञान उपयोगी हुन्छ, तर agent बनाउँदा चाहिने मुख्य कुरा यही पुस्तकमा क्रमशः बुझाइन्छ।

The official LangChain Learn page groups learning into Deep Agents, LangChain, LangGraph, multi-agent patterns, memory, and context engineering. This book follows that progression but adds beginner explanations, mental models, code, exercises, debugging advice, and complete project blueprints.

Version note
Deep Agents is evolving quickly. The official GitHub release page listed deepagents 0.7.19 around the time this edition was prepared, and the project is still pre-1.0. Pin versions for important projects and re-check the docs when upgrading.

How to study this book

1. Read Chapters 1-6 without trying to memorize code. Your goal is to build the mental model.
2. Run the minimal examples in Chapters 7-10.
3. Then learn context, memory, skills, and subagents in Chapters 11-15.
4. Build one complete project from Chapter 20 before adding many tools.
5. Use the cheat sheets and glossary whenever a term becomes confusing.

> नेपालीमा: सबै कुरा एकैपटक याद गर्न खोज्नु पर्दैन। पहिले "agent कसरी सोच्दछ/चल्छ" भन्ने flow बुझ्नुहोस्। त्यसपछि code आफैं सजिलो हुन्छ।

What you will be able to build

- A research agent that searches, reads, writes files, and prepares reports.
- A domain agent with persistent instructions and reusable skills.
- A supervisor agent that delegates specialized work to subagents.
- An agent that pauses before risky actions and asks for human approval.
- An agent connected to APIs, databases, internal systems, or MCP servers.
- A production-oriented architecture with tracing, checkpoints, permissions, evaluation, and security controls.

Contents | विषय सूची

Ch.	Topic
1	AI, LLM, Token, Prompt, Context - absolute basics
2	What makes an Agent different from a chatbot?
3	LangChain vs LangGraph vs Deep Agents
4	The Deep Agent mental model and execution loop
5	Python concepts you need
6	Environment setup: venv, uv, keys, packages
7	Your first Deep Agent
8	Messages, invoke, state, result, and threads
9	System prompts: the operating policy of your agent
10	Tools: giving the agent hands
11	Filesystem and backends
12	Context engineering and context compression
13	Memory: short-term vs long-term
14	Skills: reusable on-demand knowledge
15	Subagents and delegation
16	Middleware and task planning
17	Human-in-the-loop, permissions, and safety
18	Streaming, tracing, LangSmith, debugging
19	MCP and external integrations
20	Complete Project A: research/report agent
21	Complete Project B: bioinformatics research agent
22	Complete Project C: multi-agent business workflow
23	Testing, evaluation, cost, reliability, production
24	Common mistakes and troubleshooting
25	Learning roadmap, cheat sheet, glossary, references

1. AI, LLM, Token, Prompt, Context - Absolute Basics

1.1 What is an LLM?

A Large Language Model (LLM) is a model trained to process and generate sequences of tokens. In everyday use, you send it messages and it predicts useful next tokens until it forms an answer. It does not automatically have access to your computer, database, company files, browser, or APIs. Those capabilities must be connected.

> नेपालीमा: LLM लाई धेरै ठूलो text-processing brain जस्तो सोच्न सकिन्छ। तपाईंले प्रश्न दिनुहुन्छ, उसले text generate गर्छ। तर उसलाई स्वतः तपाईंको database, file, browser वा company system access हुँदैन। ती काम गर्न "tools" दिनुपर्छ।

1.2 Prompt

A prompt is information sent to the model. In agent systems, the prompt is not only the user question. It may include a system prompt, memory, skill descriptions, tool descriptions, previous messages, retrieved documents, and runtime metadata.

> नेपालीमा: Prompt भनेको केवल user ले टाइप गरेको प्रश्न होइन। Agent system मा system instruction, पुराना messages, memory, tools को description, retrieved document आदि सबै model को context हुन सक्छन्।

1.3 Token

Models read tokens, not characters in the human sense. A token can be a word, part of a word, punctuation, or another encoded unit. More context usually means more input tokens, which affects cost, latency, and the amount of irrelevant material the model must reason through.

1.4 Context window

The context window is the amount of information a model can consider in one model call. An agent can run much longer than a single call, but each model call still has a context limit. Deep Agents therefore use mechanisms such as summarization, file offloading, skills, and subagents to manage growing context.

> नेपालीमा: Context window लाई model को "एकपटकमा पढ्न सक्ने working desk" जस्तो सोच्नुहोस्। Desk मा अनावश्यक कागज थुप्रियो भने राम्रो काम गर्न गाह्रो हुन्छ। Deep Agents को मुख्य शक्ति यही context व्यवस्थित गर्नु हो।

1.5 Tool calling

Tool calling means the model returns a structured request such as "call search(query=...)" instead of pretending it already performed the action. The agent runtime executes the tool, returns the result to the model, and the model decides what to do next.

```
User question
   |
   v
LLM decides: answer directly OR call a tool
   |
   +--> tool call ---> Python/API/DB/File/Search
                            |
                            v
                       tool result
                            |
                            v
                       back to LLM
                            |
                            v
                       next action
```

Key idea
An LLM is the reasoning/generation component. An agent is the surrounding system that lets the LLM repeatedly observe, decide, act with tools, and continue until it reaches a stopping condition.

2. What Makes an Agent Different from a Chatbot?

2.1 Chatbot pattern

A simple chatbot receives a message and returns a message. It can be very capable, but the application may not let it independently call tools or continue through a multi-step loop.

```
User -> Model -> Answer
```

2.2 Agent pattern

An agent can choose actions. A typical loop is: read state -> call model -> model asks for tool -> execute tool -> add result to state -> call model again -> stop when the model gives a final response.

```
User
  |
  v
[State / messages]
  |
  v
[Model]
  | decides
  +-------- final answer --------> END
  |
  +-------- tool call
              |
              v
           [Tool]
              | result
              +--------> back to State -> Model
```

नेपालीमा: Chatbot ले "जवाफ" दिन्छ। Agent ले "काम" पनि गर्न सक्छ। उदाहरण: search गर्नु, file पढ्नु, database query गर्नु, report लेख्नु, अर्को subagent लाई काम दिनु।

2.3 Why agents can fail

Agents are powerful because they choose actions, but that also creates failure modes: wrong tool choice, too many tool calls, hallucinated parameters, stale context, loops, expensive runs, destructive actions, and poor delegation. Good agent engineering therefore includes tool design, context control, permissions, limits, tracing, evaluation, and human approval.

2.4 The autonomy ladder

Level	System behavior	Example
0	Text only	Question answering
1	Single deterministic tool	Always call one fixed API
2	Model chooses tools	Search, calculator, database
3	Multi-step agent loop	Search -> read -> analyze -> write
4	Deep agent	Filesystem + context management + delegation + persistence
5	Workflow of agents	Supervisor + specialists + approval + production controls

3. LangChain vs LangGraph vs Deep Agents

This distinction is the foundation. The official documentation describes Deep Agents as a standalone library built on LangChain agent building blocks and using the LangGraph runtime for durable execution, streaming, human-in-the-loop, and related runtime features.

Layer	Simple meaning	When you use it
LangChain	Agent/framework building blocks	Models, messages, tools, create_agent, middleware, integrations
LangGraph	Durable execution runtime / graph orchestration	State, checkpoints, persistence, streaming, interrupts, custom workflows
Deep Agents	Opinionated agent harness on top	Long-running multi-step agents with filesystem, context management, delegation, skills, memory

```
                Your Application
                       |
                  Deep Agents
          (opinionated agent harness)
                       |
                   LangChain
          (models, tools, agent building blocks)
                       |
                   LangGraph
        (runtime: state, persistence, streaming)
                       |
       Model providers / APIs / DB / filesystem
```

नेपालीमा: LangChain लाई “parts box”, LangGraph लाई “engine/runtime”, Deep Agents लाई “पहिल्यै धेरै उपयोगी भाग जडान गरिएको agent machine” जस्तो सम्झनुहोस्।

3.1 When should I use which?

- Use a simple LangChain create_agent when the problem is small and you want fewer built-in opinions.
- Use Deep Agents when the work is long, file-heavy, research-heavy, multi-step, or benefits from subagents, memory, and context management.
- Use LangGraph directly when you need precise custom workflow control: explicit nodes, transitions, branches, state machines, or deterministic orchestration.

3.2 Deep Agents is not a magical new model

Deep Agents does not replace the LLM. It is a harness around a tool-calling model. The quality of the model still matters, but architecture, tools, context, and constraints can matter just as much.

4. The Deep Agent Mental Model and Execution Loop

4.1 Core capabilities

The current Deep Agents docs group the harness around four broad areas: execution environment, context management, delegation/planning, and steering. Together these let the agent act, manage long context, delegate work, and remain under human control.

Capability	What it means in practice
Execution environment	Tools + virtual filesystem + optional sandbox/interpreter
Context management	Skills + memory + summarization + offloading + prompt caching
Delegation	Subagents and optional structured task planning
Steering	Human-in-the-loop, interrupts, permissions, runtime control

4.2 What create_deep_agent() gives you

You create the harness with create_deep_agent(...). You supply the model and optionally tools, a system prompt, memory files, skill directories, a backend, subagents, middleware, permissions, interrupt rules, schemas, checkpointing, and a store.

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[...],
    system_prompt="...",
    memory=["./AGENTS.md"],
    skills=["./skills/"],
)
```

4.3 Built-in harness tools

The current docs list built-in file/delegation tools including ls, read_file, write_file, edit_file, delete, glob, grep, execute (when the selected sandbox backend supports shell execution), and task for spawning subagents.

Tool	Beginner meaning
ls	See files/folders
read_file	Read a file
write_file	Create/overwrite a file
edit_file	Replace exact text in a file
delete	Remove a file/directory
glob	Find files by filename pattern
grep	Search inside files
execute	Run shell command in a supported sandbox
task	Delegate a task to a subagent

नेपालीमा: Deep Agent ले file system लाई बाहिरी memory/scratchpad जस्तो प्रयोग गर्न सक्छ। लामो search result context मा राखिरहनु भन्दा file मा लेखेर पछि आवश्यक भाग मात्र पढ्न सक्छ।

4.4 Planning is optional

Structured todo planning with write_todos is not something to assume is always enabled. The current docs say to opt in with TodoListMiddleware. This is useful for long tasks where visible planning and task tracking improve reliability.

5. Python Concepts You Need

5.1 Function

A tool can often be an ordinary Python function. Inputs become tool arguments; the return value becomes the tool result.

```
def add(a: int, b: int) -> int:
    """Add two integers."""
    return a + b
```

नेपालीमा: यहाँ function को नाम add हो। a र b input हुन्। return भएको value agent ले tool result को रूपमा पाउँछ।

5.2 Type hints and docstrings matter

Deep Agents/LangChain can infer a tool schema from a function signature and docstring. Clear names, input types, and descriptions make the model more likely to call the tool correctly.

```
def get_sample_status(sample_id: str) -> str:
    """Return the current laboratory processing status for a sample ID."""
    ...
```

5.3 List and dictionary

```
tools = [search_tool, database_tool]

config = {
```

```
    "name": "researcher",
    "description": "Research scientific literature",
    "tools": [search_tool],
}
```

Lists hold ordered items. Dictionaries hold key-value pairs. Agent configuration frequently uses both.

5.4 Environment variable

```
# Linux / macOS shell
export OPENAI_API_KEY="..."

# Python
import os
key = os.environ["OPENAI_API_KEY"]
```

Do not hard-code production API keys into source files committed to Git.

5.5 Sync vs async

invoke(...) is the common synchronous call. ainvoke(...) is the asynchronous form. You can begin with synchronous code. Learn async when your application handles many concurrent requests, MCP sessions, streaming, or parallel I/O.

6. Environment Setup: venv, uv, Keys, Packages

6.1 Recommended simple project layout

```
deep-agent-learning/
├── .env                 # secrets (do not commit)
├── .gitignore
├── main.py
├── AGENTS.md             # optional persistent instructions
└── skills/
    └── research/
        └── SKILL.md
```

6.2 Option A: venv + pip

```
mkdir deep-agent-learning
cd deep-agent-learning
python -m venv .venv
source .venv/bin/activate
pip install -U deepagents
```

6.3 Option B: uv

```
mkdir deep-agent-learning
cd deep-agent-learning
uv init
uv add deepagents
uv sync
```

The official quickstart currently supports both pip and uv. If you are already comfortable with uv, it is convenient for reproducible dependency management.

6.4 API key examples

```
export OPENAI_API_KEY="your-key"
# or provider-specific key such as GOOGLE_API_KEY / ANTHROPIC_API_KEY
```

Deep Agents require a model that supports tool calling. The official quickstart also documents Ollama for local use; the local Ollama server must be running and the selected model must support the needed tool behavior.

6.5 Freeze versions for serious work

```
pip freeze > requirements.txt
# or use uv.lock when using uv
```

Why pin versions?
Deep Agents is still pre-1.0, so minor releases may contain API changes. A validated production system should record exact package and model versions.

7. Your First Deep Agent

7.1 Minimal agent

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="openai:gpt-5.5",
    system_prompt="You are a clear and careful assistant.",
)

result = agent.invoke({
    "messages": [
        {"role": "user", "content": "Explain what an AI agent is in 5 lines."}
    ]
})

print(result["messages"][-1].content)
```

7.2 Read the code line by line

Code	Meaning
from deepagents import create_deep_agent	Import the constructor.
agent = create_deep_agent(...)	Build a compiled runnable agent graph.
model=...	Choose the tool-capable chat model.
system_prompt=...	Give high-level behavior instructions.
agent.invoke(...)	Run one agent invocation.
messages	The conversation input stored in agent state.
result["messages"][-1]	Take the final message from the resulting state.

नेपालीमा: सबैभन्दा पहिले agent “object” बनाइन्छ। त्यसपछि invoke गरेर user message पठाइन्छ। Result केवल string होइन; state भित्र messages आदि हुन सक्छन्।

7.3 A web research agent

The current quickstart shows provider-native web search. For OpenAI-style provider tools, the search tool can be passed as a provider tool dictionary.

```
from deepagents import create_deep_agent

internet_search = {"type": "web_search"}

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[internet_search],
    system_prompt=(
        "You are a research assistant. Search when fresh information is needed. "
        "Separate evidence from your interpretation."
    ),
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Research LangGraph and summarize it."}]
})
print(result["messages"][-1].content)
```

7.4 What the agent may do internally

1. Read the user request.
2. Decide that current information is needed.
3. Call the web search tool.
4. Receive search results.
5. Possibly write large material to files or delegate research to a subagent.
6. Synthesize a final answer.

8. Messages, invoke, State, Result, and Threads

8.1 Messages

Agent conversations are represented as messages. Common roles are user/human, assistant/AI, system, and tool. During a real agent run the message history may contain AI tool-call messages and tool-result messages, not only human-readable chat.

```
{
    "messages": [
        {"role": "user", "content": "Find the latest paper and summarize it."}
    ]
}
```

8.2 State

State is the working data carried through the graph execution. Messages are a major part of state, but middleware can add other state such as todo items. Checkpointing can persist state so a thread can be resumed.

> नेपालीमा: State भनेको agent को run चलिरहँदा बोकेर हिँड्ने working data हो। Messages, todo list, intermediate state आदि यसमा पर्न सक्छन्।

8.3 Thread ID

When persistence/checkpointing is configured, a thread ID identifies a conversation/work thread. Using a stable thread ID lets the runtime continue the same thread instead of starting from nothing each time.

```
config = {"configurable": {"thread_id": "user-42-session-1"}}
result = agent.invoke(input_data, config=config)
```

8.4 Short-term memory vs thread state

Do not confuse “the model remembers” with persisted state. Short-term conversational continuity comes from stored/checkpointed thread state supplied back into later runs. Long-term memory across different threads uses a store/backend strategy.

9. System Prompts: The Operating Policy of Your Agent

9.1 What belongs in a system prompt

- Role: what the agent is responsible for.
- Priorities: correctness, evidence, format, safety.
- When to use tools and when not to.
- Rules for delegation.
- Domain-specific boundaries.
- Expected output format and escalation behavior.

9.2 Bad prompt

```
You are smart. Do everything well.
```

9.3 Better prompt

```
You are a scientific research assistant.

Goals:
1. Answer using evidence.
2. Use search for claims that may have changed.
3. Distinguish source facts from your interpretation.
4. Use a subagent when a subtopic would generate large intermediate context.
5. Do not invent citations.
6. If data is missing, state the limitation.

Output:
- Summary
- Evidence
- Limitations
- Sources
```

नेपालीमा: System prompt लाई कर्मचारीको "job description + SOP + rules" जस्तो सोच्नुहोस्। धेरै vague prompt ले vague behavior दिन्छ।

9.4 Keep prompt and tools consistent

If the prompt says "always verify with database X" but the agent does not have a database tool, the instruction is impossible. Agent design is a contract among prompt, tools, context, permissions, and runtime.

10. Tools: Giving the Agent Hands

10.1 Plain Python function as a tool

```
from deepagents import create_deep_agent

def get_weather(city: str) -> str:
    """Return weather data for a city from our internal service."""
    # Replace with real API call
    return f"Weather service result for {city}"

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[get_weather],
)
```

The docs state that plain callables can be passed directly, and the tool schema can usually be inferred from the function signature and docstring.

10.2 Tool design rules

1. Give the tool one clear responsibility.
2. Use a precise function name.
3. Use typed arguments.
4. Write a docstring that explains when to use it.
5. Return compact, structured results when possible.
6. Return actionable errors instead of silently failing.
7. Do not expose a dangerous "run_anything_as_root" style tool.

10.3 Database tool pattern

```
def find_customer(customer_id: str) -> dict:
    """Look up one customer by exact customer ID."""
```

```
    # query database safely with parameterized SQL
    return {"customer_id": customer_id, "status": "active"}
```

10.4 API tool pattern

```
import requests

def get_order(order_id: str) -> dict:
    """Fetch one order from the order service."""
    r = requests.get(
        f"https://internal.example/api/orders/{order_id}",
        timeout=10,
    )
    r.raise_for_status()
    return r.json()
```

10.5 Tool output size

A common agent mistake is dumping megabytes of raw tool output into the model context. Prefer pagination, filtering, summaries, file offloading, or subagent isolation. Context engineering is not an optional afterthought for serious agents.

> नेपालीमा: Tool ले 100,000 row सबै context मा फाल्नु खराब design हुन सक्छ। आवश्यक row/filter मात्र फर्काउने, file मा राख्ने, वा subagent प्रयोग गर्ने राम्रो हुन्छ।

11. Filesystem and Backends

11.1 Why a virtual filesystem?

Long-running agents need somewhere to put notes, intermediate results, drafts, datasets, and generated artifacts. Deep Agents exposes filesystem-style tools, while a backend decides where those files actually live.

11.2 StateBackend - default idea

The current docs show StateBackend as the default backend. Think of it as a filesystem represented in agent state. It is convenient for ephemeral/agent-managed files and can participate in checkpointed state.

```
from deepagents import create_deep_agent
from deepagents.backends import StateBackend

agent = create_deep_agent(
    model="openai:gpt-5.5",
    backend=StateBackend(),
)
```

11.3 FilesystemBackend

FilesystemBackend maps agent file operations to a real filesystem root. This is useful when the agent should work with actual project files. Treat this as a security boundary: only expose directories the agent genuinely needs.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
    model="openai:gpt-5.5",
    backend=FilesystemBackend(root_dir="./workspace", virtual_mode=True),
)
```

11.4 CompositeBackend

A CompositeBackend can route different virtual paths to different storage backends. For example, ordinary scratch files can use state while /memories/ can route to a persistent store.

```
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
```

```
backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(namespace=lambda rt: ("memories",)),
    },
)
```

11.5 Backend mental model

```
Agent tool: write_file("/notes/result.md", ...)
                |
                v
             Backend
      +---------+----------+
      |         |          |
    State    Local FS    Persistent Store
```

नेपालीमा: Agent लाई path देखिन्छ। वास्तवमा त्यो path memory state मा हो कि real disk मा हो कि persistent store मा हो भन्ने backend ले निर्णय गर्छ।

12. Context Engineering and Context Compression

12.1 What is context engineering?

The Deep Agents docs define context engineering as giving the agent the right information and tools in the right format so it can accomplish tasks reliably. This is one of the most important ideas in modern agent engineering.

नेपालीमा: Context engineering भनेको "model लाई धेरै कुरा दिने" होइन; "ठिक समयमा ठिक कुरा दिने" हो।

12.2 Five context categories

Context type	Meaning
Input context	System prompt, memory, skills, tool instructions available at start
Runtime context	Per-run metadata such as user ID, API connections, feature flags
Context compression	Summarize/offload when context becomes too large
Context isolation	Delegate heavy work to a subagent with its own context
Long-term memory	Persistent information available across conversations

12.3 Why "maximum context" is not always best

A huge prompt can contain more facts but also more noise, conflicting instructions, stale data, repeated tool output, and higher token cost. Good context engineering maximizes relevance, not raw size.

12.4 Practical context strategy

1. Keep stable global rules in system prompt or memory.
2. Keep specialized procedures in skills and load them only when relevant.
3. Retrieve specific domain data only when needed.
4. Write large intermediate data to files instead of repeating it in messages.
5. Delegate context-heavy subtasks to subagents.
6. Summarize older conversation when necessary.

13. Memory: Short-Term vs Long-Term

13.1 Short-term memory

The current memory docs distinguish short-term memory inside a thread from long-term memory across threads. Short-term conversation history and scratch state are managed as agent state/checkpoints.

13.2 Long-term memory

Long-term memory persists across separate conversations. Deep Agents can load configured AGENTS.md files at startup. Persistent memory typically requires a durable store/backend rather than only in-process memory.

13.3 AGENTS.md example

```
# AGENTS.md

## Response style
- Use concise technical language.
- Show runnable Python examples.
- State uncertainty clearly.

## Domain rules
- Never invent laboratory validation data.
- Separate observed evidence from future recommendations.
```

Then configure it:

```
agent = create_deep_agent(
    model="openai:gpt-5.5",
    memory=["./AGENTS.md"],
)
```

13.4 What should go into memory?

Good memory	Bad memory
Stable user/team preference	Every temporary chat detail
Durable project conventions	Huge raw logs
Persistent domain rules	Secrets copied into plain text unnecessarily
Frequently needed background	Information that should be fetched fresh from source

नेपालीमा: Memory मा “सधैं उपयोगी हुने स्थायी जानकारी” राख्नु राम्रो हो। आजको temporary result, huge log, वा frequently-changing data लाई memory बनाउनु राम्रो होइन।

14. Skills: Reusable On-Demand Knowledge

14.1 Skill vs memory vs tool

The official skills docs make a useful distinction: skills are on-demand capabilities discovered through progressive disclosure; memory is persistent context loaded at startup; tools are programmatic actions available to call.

Concept	Use it for	Loaded when
Skill	Procedure / specialized instructions / references	When relevant
Memory	Stable persistent context	At startup
Tool	Action: API, DB, file, search	Callable during run

14.2 Folder structure

```
skills/
└── literature-review/
    ├── SKILL.md
    ├── references/
    │   └── evidence-grading.md
    ├── scripts/
```

```
    │   └── normalize_doi.py
    └── assets/
        └── report_template.md
```

14.3 SKILL.md example

```
---
name: literature-review
description: Review scientific literature and produce an evidence table.
---

# Literature Review Skill

Use this skill when the user asks for a literature review.

Workflow:
1. Clarify the clinical/scientific question from the request.
2. Search authoritative sources.
3. Extract study design, population, endpoint, result, limitation.
4. Separate primary evidence from commentary.
5. Write the final evidence table and narrative summary.

For grading rules, read references/evidence-grading.md.
```

14.4 Configure skills

```
agent = create_deep_agent(
    model="openai:gpt-5.5",
    skills=["./skills/"],
)
```

14.5 Why skills save context

If you put every specialized procedure into the system prompt, every request pays the token and attention cost. Skills let the agent discover and read a procedure only when the task needs it.

> नेपालीमा: Skills भनेको ठूलो manual लाई सधैं prompt मा राख्नुको सट्टा आवश्यक chapter मात्र खोल्ने तरिका हो। यही progressive disclosure हो।

15. Subagents and Delegation

15.1 Why subagents?

The current docs explicitly highlight context bloat as a reason for subagents. A subagent performs detailed work in an isolated context and returns a concise result to the supervisor. The main agent does not need to carry every intermediate search/tool result.

```
Supervisor Deep Agent
   |
   +--> Research subagent ----> searches 12 sources ----> summary
   |
   +--> Data subagent --------> analyzes CSV ----------> metrics
   |
   +--> Writer subagent ------> drafts report ---------> draft
   |
   +--> Supervisor combines results
```

> नेपालीमा: Subagent भनेको अर्को छुट्टै worker हो। Supervisor ले “यो research तिमी गर” भनेर task दिन्छ। Worker ले आफ्नो context मा धेरै काम गर्छ र छोटो result फर्काउँछ।

15.2 Custom subagent

```
researcher = {
    "name": "researcher",
    "description": "Research scientific and technical sources.",
    "system_prompt": (
        "You are a careful researcher. Return concise evidence with source details."
```

```
    ),
    "tools": [internet_search],
    "model": "openai:gpt-5.5",
}

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[internet_search],
    subagents=[researcher],
)
```

15.3 When to delegate

- A subtask requires many tool calls.
- A subtask has a specialized role or prompt.
- You want context isolation.
- Different model/tool sets are better for different work.
- You want independent parallel or asynchronous workstreams (using the appropriate async subagent pattern).

15.4 When not to delegate

- The task is tiny and one tool call is enough.
- The supervisor needs every detail from the subtask anyway.
- Delegation overhead costs more than it saves.

15.5 Skills inheritance

Current docs note that the general-purpose subagent can inherit skills from the main agent, while custom subagents do not inherit skills by default. Give custom subagents their own skills when needed.

16. Middleware and Task Planning

16.1 What is middleware?

Middleware is logic inserted around the agent/model/tool lifecycle. It can change prompts, summarize context, retry tools, limit calls, require approval, add todo planning, select tools, or implement custom behavior.

```
User request
   |
[before_agent middleware]
   |
[before_model / wrap_model_call]
   |
[MODEL]
   |
[after_model]
   |
[tool middleware] -> TOOL
   |
[loop]
   |
[after_agent]
```

16.2 TodoListMiddleware

The current LangChain middleware adds write_todos when you opt in to TodoListMiddleware. This gives the agent structured task planning and todo state.

```
from deepagents import create_deep_agent
from langchain.agents.middleware import TodoListMiddleware

agent = create_deep_agent(
    model="openai:gpt-5.5",
```

```
    middleware=[TodoListMiddleware()],
)
```

16.3 Useful middleware categories

Middleware idea	Why you might use it
Summarization	Compress old messages near context limits
Human-in-the-loop	Pause before sensitive tool calls
Model/tool call limits	Prevent runaway loops or cost
Fallback/retry	Recover from provider or tool failures
PII handling	Detect/redact sensitive personal data
Tool selector	Reduce a huge tool set to relevant tools
Todo planning	Track multi-step tasks

16.4 Do not add middleware blindly

Every middleware layer changes agent behavior and can complicate debugging. Start with a minimal agent, measure a failure mode, then add the specific middleware that addresses it.

17. Human-in-the-Loop, Permissions, and Safety

17.1 Why human approval exists

Agents can call tools that modify files, send messages, trigger payments, update records, or run code. For actions with real-world consequences, a human approval step is often appropriate.

17.2 interrupt_on pattern

```
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model="openai:gpt-5.5",
    interrupt_on={
        "write_file": True,
    },
    checkpointer=MemorySaver(),
)
```

More advanced interrupt rules can allow approve/edit/reject decisions conditionally, for example only when a write targets a path outside an allowed workspace.

17.3 Filesystem permissions

Treat paths as capabilities. Give the agent read/write access only where needed. A research agent may need write access to ./reports but not to ~/.ssh, production secrets, or the entire server filesystem.

17.4 Sandbox for code execution

If an agent can execute shell commands or arbitrary code, use an isolated sandbox appropriate to your risk model. Do not equate “AI tool” with “safe tool.” The tool executes with whatever privileges your runtime grants it.

> नेपालीमा: Agent लाई shell/root जस्तो शक्तिशाली access दिँदा LLM मात्र होइन, पूर्ण execution environment को security सोच्नुपर्छ। Least privilege महत्वपूर्ण छ।

17.5 Security checklist

- Least privilege tools and filesystem paths.
- Secrets in secret managers/environment, not prompts/logs when avoidable.

- Parameter validation at the tool boundary.
- Approval before destructive/financial/external actions.
- Rate limits and call limits.
- Audit logs/traces.
- Sandbox code execution.
- Test prompt injection from untrusted documents/web pages.

18. Streaming, Tracing, LangSmith, and Debugging

18.1 Why tracing matters

When an ordinary function fails, you inspect a stack trace. When an agent gives a bad answer, you often need to inspect model calls, tool calls, tool results, delegation, and state transitions. LangSmith tracing is designed to provide this visibility.

18.2 LangSmith environment variables

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="your-langsmith-api-key"
```

18.3 What to inspect

1. What exact user input reached the agent?
2. What system prompt/context was active?
3. Which tool did the model select?
4. Were tool arguments correct?
5. What did the tool actually return?
6. Did context become huge or irrelevant?
7. Was a subagent given the right task?
8. Did the agent loop unnecessarily?
9. Which step consumed most tokens/time?

18.4 Streaming

Deep Agents use LangGraph streaming capabilities so applications can surface model output, tool events, and delegated work progressively. Streaming improves UX and debugging for long runs because users do not have to see only a final opaque result.

19. MCP and External Integrations

19.1 What is MCP?

Model Context Protocol (MCP) is an open protocol for exposing tools/resources from external services through a standardized interface. Deep Agents can consume tools from MCP servers through LangChain MCP integration.

> नेपालीमा: MCP लाई “agent र external system बीच standardized plug/socket” जस्तो सोच्न सकिन्छ। हरेक integration का लागि नयाँ custom glue code लेख्नुपर्ने काम कम हुन सक्छ।

19.2 Install

```
pip install "langchain[mcp]"
```

19.3 Simplified MCP pattern

```
import asyncio
from deepagents import create_deep_agent
from langchain.mcp import MCPAdapter

async def main():
    config = {
        "mcpServers": {
            "my_server": {"url": "http://localhost:8000/mcp"}
        }
    }

    async with MCPAdapter(config) as adapter:
        tools = await adapter.list_tools()
        agent = create_deep_agent(
            model="openai:gpt-5.5",
            tools=tools,
        )
        result = await agent.ainvoke({
            "messages": [{"role": "user", "content": "Use the MCP server to help me."}]
        })
        print(result["messages"][-1].content)

asyncio.run(main())
```

19.4 MCP is not automatically trustworthy

An MCP server is still an external capability. Authenticate it, restrict permissions, inspect exposed tools, validate parameters, and treat untrusted tool output as untrusted content.

20. Complete Project A: Research and Report Agent

20.1 Goal

Build an agent that can research a topic, keep large intermediate findings out of the main context when possible, organize work with todos, delegate deep research, and write a report file.

20.2 Project layout

```
research-agent/
├── main.py
├── AGENTS.md
├── workspace/
└── skills/
    └── report-writing/
        └── SKILL.md
```

20.3 AGENTS.md

```
# Persistent agent instructions

- Prefer primary or authoritative sources.
- Distinguish facts from analysis.
- Never invent citations.
- For long research, delegate a bounded subtask to a researcher.
- Save final reports under /reports/.
```

20.4 Skill

```
---
name: report-writing
description: Turn research findings into a concise evidence-based report.
---

# Report writing

Use this skill for final written reports.
Structure:
```

```
1. Executive summary
2. Question and scope
3. Findings
4. Evidence table
5. Limitations
6. Sources
```

20.5 Main code

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langchain.agents.middleware import TodoListMiddleware

web_search = {"type": "web_search"}

researcher = {
    "name": "researcher",
    "description": "Investigate one bounded research question and return evidence.",
    "system_prompt": (
        "Research carefully. Prefer authoritative sources. "
        "Return concise findings, evidence, and limitations."
    ),
    "tools": [web_search],
    "model": "openai:gpt-5.5",
}

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[web_search],
    system_prompt=(
        "You are the supervisor research agent. Plan complex work, "
        "delegate context-heavy research, and produce a final report."
    ),
    memory=["./AGENTS.md"],
    skills=["./skills/"],
    subagents=[researcher],
    middleware=[TodoListMiddleware()],
    backend=FilesystemBackend(root_dir="./workspace", virtual_mode=True),
)

request = {
    "messages": [{
        "role": "user",
        "content": (
            "Research how context engineering improves long-running AI agents. "
            "Write a report to /reports/context-engineering.md."
        ),
    }]
}

result = agent.invoke(request)
print(result["messages"][-1].content)
```

20.6 What to observe

- Does the model create a sensible plan?
- Does it delegate only when needed?
- Does the researcher return concise results?
- Does the agent use files rather than keeping everything in chat?
- Does the final report cite sources rather than inventing them?

21. Complete Project B: Bioinformatics Research Agent

This project demonstrates how to turn a general Deep Agent into a domain-specific scientific assistant without putting every protocol into one giant prompt.

21.1 Architecture

```
                Supervisor
                    |
      +-------------+--------------+
      |             |              |
Literature      Variant/NGS     Report writer
researcher      specialist      skill
      |             |
search tools    internal tools
      |             |
      +------ evidence ----------+
                    |
              final synthesis
```

21.2 Domain tools

```
def lookup_internal_sop(sop_id: str) -> str:
    """Return an approved internal SOP excerpt by exact SOP identifier."""
    ...

def query_variant_db(query: str) -> list[dict]:
    """Query an approved internal variant knowledge database."""
    ...

def fetch_validation_record(validation_id: str) -> dict:
    """Fetch one validation record from the controlled validation repository."""
    ...
```

21.3 Critical scientific prompt rule

```
Scientific integrity rules:
- Do not manufacture sample counts, sensitivity, specificity, QC metrics, or validation outcomes.
- Clearly label observed historical evidence versus a proposed future check.
- Use the controlled internal tool for internal evidence.
- If a record is missing, say it is missing.
- External publications do not replace internal validation evidence.
```

नेपालीमा: Scientific/medical agent मा confident-looking hallucination धेरै खतरनाक हुन सक्छ। “थाहा छैन”, “record भेटिएन”, “future confirmation चाहिन्छ” भन्न सक्ने design राम्रो हो।

21.4 Why skills are useful here

Separate specialized procedures into skills: literature-review, CAP-validation-review, NIPT-terminology, report-formatting, or variant-evidence. The main supervisor sees only the relevant skill when needed, reducing prompt bloat.

21.5 Human approval points

- Before changing an approved validation document.
- Before sending a clinical-facing report externally.
- Before writing to a production LIMS/EMR endpoint.
- Before destructive data modification.

22. Complete Project C: Multi-Agent Business Workflow

22.1 Example: global marketing / buyer research

A business workflow can use one supervisor plus specialists for market research, lead enrichment, email drafting, and CRM updates. The supervisor should not directly receive every raw search page or API response.

```
Supervisor
  |
  +-- country-researcher -> market facts
  +-- buyer-finder ------> candidate companies
  +-- verifier ----------> validate website/contact evidence
```

```
  +-- email-writer ------> draft only
  +-- CRM tool ----------> write after approval
```

22.2 Separation of powers

A useful production pattern is to separate “research and propose” from “commit a real-world action.” For example, one agent can draft an email, while a human approves the draft before a sending tool is allowed to execute.

22.3 Example subagent configuration

```
buyer_finder = {
    "name": "buyer-finder",
    "description": "Find plausible distributor or laboratory buyers in one country.",
    "system_prompt": (
        "Return candidates with evidence URLs. Do not invent email addresses. "
        "Mark unverified contact details as unverified."
    ),
    "tools": [web_search],
}

email_writer = {
    "name": "email-writer",
    "description": "Draft personalized B2B outreach from verified lead facts.",
    "system_prompt": (
        "Draft only. Do not send. Do not claim facts absent from the supplied lead record."
    ),
}
```

22.4 Production data model idea

Table	Purpose
leads	Company, country, status, evidence
contacts	Person/email/source/verification
research_runs	Trace of research job
drafts	Generated email drafts
approvals	Human approve/reject/edit record
send_log	Actual transmission audit
agent_config	Model/prompt/tool/version configuration

23. Testing, Evaluation, Cost, Reliability, Production

23.1 Agent testing is not only “did code run?”

A program can execute without exceptions and still be a bad agent. You need behavioral evaluation: correct tool choice, factual accuracy, safe action boundaries, appropriate delegation, output format, latency, token usage, and recovery from tool failures.

23.2 Build a small evaluation set

Test type	Example
Happy path	Research a normal supported topic
No-data case	Internal record does not exist
Tool failure	API returns timeout
Prompt injection	A webpage tells agent to reveal secrets
Permission test	Agent tries to write outside workspace
Long context	Large tool outputs trigger context pressure
Delegation test	Complex task should use researcher
Simple task	Tiny request should not over-delegate

23.3 Metrics

- Task success rate.
- Grounded/cited factual accuracy.
- Tool call accuracy and error rate.
- Average model calls per task.
- Average tool calls per task.
- Latency and token cost.
- Human override/rejection rate.
- Safety/permission violations.

23.4 Production architecture

```
Client/UI
   |
API layer (auth, rate limit)
   |
Agent service
   |-- Deep Agent graph
   |-- approved tools
   |-- checkpointer
   |-- persistent store
   |-- sandbox / filesystem permissions
   |
External systems: DB / LIMS / CRM / search / MCP
   |
Observability: traces + logs + evals + alerts
```

23.5 Version everything

- Python package versions.
- Model provider + model name/version where available.
- System prompt revision.
- Skill revision.
- Tool/API schema revision.
- Evaluation dataset revision.
- Deployment configuration.

24. Common Mistakes and Troubleshooting

Symptom	Likely cause	What to check
Agent never calls tool	Tool description unclear or model lacks tool calling	Model capability, tool name/docstring, prompt
Wrong tool arguments	Schema ambiguous	Types, descriptions, validation
Very expensive run	Loops / too much context / over-delegation	Tracing, call limits, context size
Hallucinated facts	No authoritative tool/retrieval or weak grounding	Evidence workflow, source requirements
Context too large	Raw tool output and long history	Offload, summarize, subagents, skills
Subagent gives poor result	Task too vague	Bound scope, explicit expected output
Memory not persistent	Using state/in-memory components only	Durable store/checkpointer/backend
Agent changes dangerous file	Permissions too broad	Filesystem root, permission rules, HITL
Infinite/long loop	No stopping pressure or repeated tool failure	Tool errors, model/tool call limits
Works locally, fails production	Secrets/path/network/provider differences	Environment parity and tracing

24.1 Debugging sequence

1. Reproduce with the smallest failing input.
2. Inspect trace: model request -> model response -> tool call -> tool result.
3. Check whether the model had the right context at that moment.
4. Check tool schema and actual tool response.
5. Remove unnecessary middleware/subagents temporarily.
6. Add deterministic validation at tool boundaries.
7. Only then adjust the prompt.

नेपालीमा: Agent खराब हुँदा prompt मात्र बदल्ने बानी नगर्नुहोस्। धेरै पटक समस्या tool schema, wrong data, context, permission, timeout, वा architecture मा हुन्छ।

24.2 Ten beginner mistakes

- Putting the whole company manual in the system prompt.
- Giving the agent dozens of vaguely named tools on day one.
- Using a very powerful shell tool instead of narrow safe functions.
- Assuming memory automatically persists across restarts.
- Assuming a subagent automatically inherits every custom skill/tool.
- Not checking tool calling support for the chosen model.
- Logging secrets and private data into traces without a policy.
- Letting the agent send/write/delete without approval where approval is needed.
- No evaluation dataset - judging quality only by a few demos.
- Upgrading agent libraries in production without pinning/testing.

25. Learning Roadmap, Cheat Sheet, Glossary, References

25.1 30-day learning roadmap

Days	Focus	Deliverable
1-3	LLM, messages, tools, agent loop	One simple tool-calling agent
4-7	create_deep_agent + filesystem	Agent writes a report file
8-10	System prompts + tool design	3 narrow reliable tools
11-14	Context engineering + memory + skills	AGENTS.md + one SKILL.md
15-18	Subagents	Supervisor + researcher
19-21	Middleware + planning + limits	Todo planning and limits
22-24	HITL + permissions + sandbox	Approval before risky action
25-27	LangSmith + evaluation	10-case eval set
28-30	MCP / real internal integration	Small production-style prototype

25.2 One-page mental cheat sheet

```
MODEL      = reasoning/generation engine
TOOL       = action the model may request
AGENT      = model + tool loop + state
LANGCHAIN  = agent/model/tool building blocks
LANGGRAPH  = durable stateful runtime / graph
DEEP AGENT = opinionated harness for long-running work

SYSTEM PROMPT = global behavior instructions
MEMORY        = persistent startup context (e.g., AGENTS.md)
SKILL         = specialized knowledge loaded on demand (SKILL.md)
BACKEND       = where agent filesystem operations are stored
CHECKPOINTER  = persists thread/run state for resume
STORE         = persistent data across threads
```

```
SUBAGENT      = isolated worker for delegated task
MIDDLEWARE    = logic around model/tool/agent lifecycle
HITL          = pause for human approval
MCP           = standardized external tool/service connection
LANGSMITH     = tracing/evaluation/observability platform
```

25.3 create_deep_agent parameter map

Parameter	Think of it as
model	Brain
system_prompt	Job description / policy
tools	Hands and external actions
memory	Always-loaded durable instructions/context
skills	On-demand specialized manuals
backend	Agent filesystem storage implementation
permissions	Filesystem access policy
subagents	Specialist workers
middleware	Interceptors / extra capabilities
interrupt_on	Human approval gates
response_format	Structured final-output schema
state_schema	Extra working state
context_schema	Per-run metadata/context
checkpointer	Thread persistence/resume
store	Cross-thread persistent storage
debug	Developer diagnostics

25.4 Glossary

Term	Meaning
Agent	A system that lets a model repeatedly choose actions/tools and continue through a task.
Agent harness	A packaged environment around the core agent loop with tools, context, memory, delegation, safety, etc.
Context	Information available to a model during a call.
Context engineering	Selecting, structuring, compressing, and routing the right information/tools for the task.
Context isolation	Keeping detailed work in a separate subagent context.
Checkpoint	Saved graph/thread state used to resume execution.
Store	Persistent storage across conversations/threads.
Tool calling	Model requests a structured function/tool execution.
Backend	Implementation behind virtual filesystem operations.
State	Data carried through an agent graph execution.
Thread	A continuing unit of conversation/work state.
Subagent	A delegated worker agent with its own context.
Middleware	Lifecycle logic around model/tool/agent calls.
Skill	A reusable procedure/knowledge package loaded on demand.
Memory	Persistent information loaded or retrieved for future conversations.
MCP	Protocol for connecting agent applications to external tool/resource servers.
HITL	Human-in-the-loop approval/steering.
Prompt injection	Untrusted content attempting to manipulate agent instructions or tool use.
Sandbox	Isolated execution environment used to reduce risk from code/shell execution.
Tracing	Recording agent calls, tool usage, state, and timings for

	debugging/observability.

25.5 Official references used for this edition

Resource	URL
LangChain Learn	https://docs.langchain.com/oss/python/learn
Deep Agents overview	https://docs.langchain.com/oss/python/deepagents/overview
Deep Agents quickstart	https://docs.langchain.com/oss/python/deepagents/quickstart
Deep Agents customization	https://docs.langchain.com/oss/python/deepagents/customization
Deep Agents tools	https://docs.langchain.com/oss/python/deepagents/tools
Deep Agents backends	https://docs.langchain.com/oss/python/deepagents/backends
Deep Agents context engineering	https://docs.langchain.com/oss/python/deepagents/context-engineering
Deep Agents memory	https://docs.langchain.com/oss/python/deepagents/memory
Deep Agents skills	https://docs.langchain.com/oss/python/deepagents/skills
Deep Agents subagents	https://docs.langchain.com/oss/python/deepagents/subagents
Deep Agents human-in-the-loop	https://docs.langchain.com/oss/python/deepagents/human-in-the-loop
LangChain agents	https://docs.langchain.com/oss/python/langchain/agents
LangGraph overview	https://docs.langchain.com/oss/python/langgraph/overview
TodoListMiddleware reference	https://reference.langchain.com/python/langchain/agents/middleware/todo/TodoListMiddleware
Deep Agents GitHub	https://github.com/langchain-ai/deepagents

All API examples should be re-checked against the live documentation when you upgrade packages. Model names and provider-specific search-tool formats may change over time.

25.6 Final learning principle

The most important transition is this: do not think of an agent as “a prompt that is smarter.” Think of it as a software system in which a probabilistic model operates inside a deliberately engineered environment of tools, state, context, memory, permissions, delegation, persistence, and observability.

> नेपालीमा: Deep Agent सिक्दा मुख्य कुरा prompt लेख्नु मात्र होइन। यो पूर्ण software architecture हो: model + tools + state + context + memory + skill + subagent + permissions + tracing। यी भागको काम छुट्टाछुट्टै बुझ्नुभयो भने complex agent पनि क्रमशः सजिलो हुन्छ।

Appendix A. Hands-on Exercises

Exercise	Task
Exercise 1 - Calculator tool	Create add, subtract, multiply tools. Ask the agent a mixed arithmetic question. Inspect which tools it calls.
Exercise 2 - File worker	Give the agent a FilesystemBackend limited to a workspace. Ask it to create notes.md and then summarize it.
Exercise 3 - Memory	Create AGENTS.md with a stable response convention. Verify it applies across multiple invocations.
Exercise 4 - Skill	Create a SKILL.md for writing bug reports. Ask an unrelated question, then a bug-report question. Observe when the skill becomes relevant.
Exercise 5 - Subagent	Create a researcher subagent. Give the supervisor a question that requires many searches. Compare context/traces with and without delegation.
Exercise 6 - Human approval	Interrupt on write_file. Ask the agent to modify a file and practice approve/edit/reject flows.
Exercise 7 - Failure handling	Make a tool that intentionally raises an exception sometimes. Add clear error handling or retry middleware and observe behavior.
Exercise 8 - Evaluation	Create 10 fixed prompts with expected behavioral requirements. Run them after every major prompt/tool change.

Appendix B. Minimal Templates You Can Copy

B.1 Minimal safe domain agent

```
from deepagents import create_deep_agent

def lookup_record(record_id: str) -> dict:
    """Fetch a single approved record by exact identifier."""
    # validate and fetch
    return {"id": record_id}

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[lookup_record],
    system_prompt=(
        "Use tools for factual record claims. "
        "Never invent missing records. "
        "State uncertainty explicitly."
    ),
)
```

B.2 Supervisor + specialist

```
specialist = {
    "name": "specialist",
    "description": "Handle detailed domain research.",
    "system_prompt": "Return concise verified findings.",
    "tools": [lookup_record],
}

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[lookup_record],
    subagents=[specialist],
)
```

B.3 Production-minded checklist

- [ ] Pin package versions

- [ ] Use secret management
- [ ] Restrict tool permissions
- [ ] Use a sandbox for code execution
- [ ] Add human approval for consequential actions
- [ ] Configure durable persistence where needed
- [ ] Trace all nontrivial runs
- [ ] Build an evaluation dataset
- [ ] Log tool errors without leaking secrets
- [ ] Review prompt injection exposure
- [ ] Record model/prompt/skill versions
- [ ] Test upgrades before production

Appendix C. What to Learn Next

After this book, the best next step is not to read every API page. Build one real agent. When you encounter a concrete limitation, study the relevant layer: LangChain tools/middleware, LangGraph state/checkpointing, Deep Agents context/backends/subagents, or MCP integrations.

> नेपालीमा: एकदम advanced theory मा जानु भन्दा एउटा वास्तविक project बनाउनुहोस्। Problem आएपछि targeted documentation पढ्नु धेरै प्रभावकारी हुन्छ।